CSM GLEAMING Solutions

COMPUTER SCIENCE MENTORS 61A

November 5, 2026 – October 9, 2026

1 Gleam basics

Gleam is a functional programming language. Functional programming is a paradigm where programs are constructed by applying and composing functions, and function definitions are trees of expressions that map values to other values, rather than a sequence of statements which update the running state of the program.

Gleam is a statically-typed, type-safe language. Type safety is the extent to which a programming language discourages or prevents type errors. The behaviors classified as type errors by a given programming language are usually those that result from attempts to perform operations on values that are not of the appropriate data type, e.g. trying to add a string to an integer.

Type enforcement can be static (catching potential errors at compile time), dynamic (associating type information with values at run-time and consulting them as needed to detect imminent errors), or a combination of both. In the context of static type systems, type safety usually involves a guarantee that the eventual value of any expression will be a legitimate member of that expression's static type.

When writing functions that operate on a given input, Gleam requires that you handle any theoretically possible output case, even if you've potentially handled such a case earlier. For instance, the function `list.max` (which behaves very similarly to Python's `max` function) does not return an integer directly, but a `Result`. This handles the case in which an empty list has been passed into `max`: it returns `Ok(value)` or `Error(Nil)` regardless of whether you've already handled the empty case earlier in your code, and are thus guaranteeing that it will always be an `Ok` result.

When writing a `case` statement in Gleam, you must handle every theoretically possible case, even if that case may not be possible in the context of your own code. For instance, say you have a function that exclusively outputs even integers, and you use one of those integers `n` in a `case` statement:

```
case n % 2 {
    0 -> //some code for the case in which n is even
    1 -> //some code for the case in which n is odd
}
```

Gleam will require you to include the latter case, in which `n % 2 == 1`, regardless of whether your integer `n` could possibly be odd.

Created by Cadence Loh and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

1. Implement `factorial`!

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> _______ * ___________________________
    False -> _______
  }
}
```

```
pub fn factorial(n: Int) -> Int {
  case n > 0 {
    True -> n * factorial(n - 1)
    False -> 1
  }
}
```

In regular recursion, our very first function will open a new frame, and each new frame will open new frames until we reach our base case. Typically, every frame must wait for subsequent frames to finish evaluating and return before they can return. That means that right before the base case, we have a lot of calls awaiting the call underneath it before doing something. Because each frame takes up physical space on our computer, recursion can be space inefficient. One way of solving this is tail recursion. If we know that we can write our recursive function in such a way that a previous frame doesn't need to do anything with the later frames, we can close previous frames once we're done with them- saving space on our computer. Here's an example:

```
pub fn main() {
  echo factorial(5)
  echo factorial(7)
}

pub fn factorial(x: Int) -> Int {
  // The public function calls the private tail recursive function
  factorial_loop(x, 1)
}

fn factorial_loop(x: Int, accumulator: Int) -> Int {
  case x {
    0 -> accumulator
    1 -> accumulator

    // The last thing this function does is call itself
    // In the previous lesson the last thing it did was multiply two ints
    _ -> factorial_loop(x - 1, accumulator * x)
  }
  //credit to the Gleam language website for this example
}
```

We've defined this helper function such that it checks our base condition and then immediately returns a recursive call. Because it doesn't do anything else with the result of the recursive call, we have no reason to keep the original frame open, and it will close the first frame to save computer memory. Ultimately, the very last recursive call will be returned with our correct value. Tail recursion only works when a language is tail-call optimized, meaning that the language knows to close unnecessary recursive frames. Python isn't tail-call optimized, so even if you write a tail recursive function, it won't be any more memory efficient. However, Gleam is tail-call optimized, so we can take advantage of this!

2. Implement `sum`, a tail recursive function that returns the sum of all elements of a list of integers.

```
fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> _______
    [first] -> _______ + _______
    [first, ..rest] -> ___________________________________
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(_______, _______)
}
```

```
fn sum_helper(l: List(Int), curr: Int) -> Int {
  case l {
    [] -> curr
    [first] -> curr + first
    [first, ..rest] -> sum_helper(rest, curr + first)
  }
}

pub fn sum(l: List(Int)) -> Int {
  sum_helper(l, 0)
}
```

Generic programming is a style of computer programming in which algorithms are written in terms of data types to-be-specified-later that are then instantiated when needed for specific types provided as parameters. This works by using a type variable instead of specifying a concrete type. It stands in for whatever specific type is being used when the function is called. These type variables are written with a lowercase name. Here are some examples:

```
pub fn main() {
  let add_one = fn(x) { x + 1 }
  let exclaim = fn(x) { x <> "!" }

  // Invalid, Int and String are not the same type
  // twice(10, exclaim)

  // Here the type variable is replaced by the type Int
  echo twice(10, add_one)

  // Here the type variable is replaced by the type String
  echo twice("Hello", exclaim)
}

// The name `value` refers to the same type multiple times
fn twice(argument: value, my_function: fn(value) -> value) -> value {
  my_function(my_function(argument))
}
//credit to the Gleam language website for this example
```

Output: 12 Hello!!

We also don't have to restrict ourselves to only doing this with functions! We can do this in type definitions as well (see the Tree type in Gleam).

3. Implement `reduce`, a tail recursive function that applies a two-argument function `f` on the elements of a list from left to right, starting with `start`.

```
fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> _______
    [first] -> ___________________________________
    [first, ..rest] -> ___________________________________
  }
}
```

```
fn reduce(l: List(a), f: fn(a, a) -> a, start: a) -> a {
  case l {
    [] -> start
    [first] -> f(start, first)
    [first, ..rest] -> reduce(rest, f, f(start, first))
  }
}
```

Just like in Python, a `Tree` in Gleam contains a `label` and a list of `Trees` as `branches`. `label` can be of a generic type, called `a` in our implementation. `leaf` returns a `Tree` with label `label` and an empty list for `branches`.

```
pub type Tree(a) {
    Tree(label: a,
        branches: List(Tree(a)))
}

pub fn leaf(label: a) -> Tree(a) {
    Tree(label, [])
}
```

4. Implement `prune_even`, which prunes every branch in a tree that has an even label. Odd child branches of branches with even labels are not preserved.

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

    let remove_even = list.filter(branches, ____________________________)
    let prune_children = list._________(____________________________________)

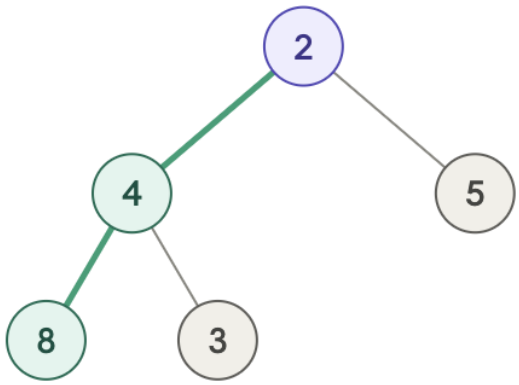
    Tree(label, prune_children)
}
```

```
pub fn prune_even(tree: Tree(Int)) -> Tree(Int) {
    let Tree(label, branches) = tree

    let remove_even =
        list.filter(branches, fn(branch) { branch.label % 2 != 0 })
    let prune_children = list.map(remove_even, prune_even)

    Tree(label, prune_children)
}
```

5. A *treemil* is a tree in which at least one of the child node(s) of each non-leaf node has a label that is a multiple of the label of the node. Alternatively, a treemil is a tree where every node has at least one path from the node to a leaf consisting entirely of nodes that are multiples of its parent. Implement `is_treemil`, which returns whether a tree is a treemil.




```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    // valid if:
    // 1. at least one child node is a multiple of its parent
    // 2. all children are valid treemils
    // leaves are valid treemils

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(label, branches) -> {
            let child_multiples = list._______(_______, _________________)
            case list._________(child_multiples) >= 1 {
                True -> list._________(___________________________________)
                False -> _________
            }
        }
    }
}
```

```
pub fn is_treemil(tree: Tree(Int)) -> Bool {
    case tree {
        Tree(_, []) -> True

        Tree(label, branches) -> {
            let child_multiples =
                list.filter(branches, fn(b) { b.label % label == 0 })
            case list.length(child_multiples) >= 1 {
                True -> list.all(list.map(branches, is_treemil), fn(x) { x
                    })
                False -> False
            }
        }
    }
}
```

6. Implement `max_treemil` which takes in a tree `t` and determines the highest number of nodes in a treemil that is a subtree of `t`. You may assume `is_treemil` is implemented correctly from the previous question.

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    //Given a tree (that is not necessarily a treemil), returns
    //the highest number of nodes in a treemil in the tree.

    case tree {
        //is a leaf
        Tree(_, []) -> _________

        //is not a leaf
        Tree(_, branches) -> {
            case ___________________________________ {
                True -> 1 + int._________(___________________________________)
                False -> result.unwrap(list.max(_________, int.compare), 0)
            }
        }
    }
}
```

```
pub fn max_treemil(tree: Tree(Int)) -> Int {
    case tree {
        Tree(_, []) -> 1

        Tree(_, branches) -> {
            case is_treemil(tree) {
                True -> 1 + int.sum(list.map(branches, max_treemil))
                False -> {
                    let sizes = list.map(branches, max_treemil)
                    result.unwrap(list.max(sizes, int.compare), 0)
                }
            }
        }
    }
}
```